

```
+ Code + Text

[1] ! pip install kaggle
    from google.colab import drive
    drive.mount('/content/drive')
    ! mkdir ~/.kaggle
    ! cp kaggle.json ~/.kaggle
    ! chmod 600 ~/.kaggle/kaggle.json
    ! kaggle datasets download 'paultimothymooney/breast-histopathology-images'
    ! unzip breast-histopathology-images.zip

inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1751_y951_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y1001_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y1051_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y601_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y651_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y701_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y751_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y801_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y851_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y901_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y951_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y1001_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y1051_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y601_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y651_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y701_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y751_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y801_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y851_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y901_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y1001_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y1051_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y601_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y651_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y701_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y751_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y801_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y851_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y901_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y951_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y1001_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y1051_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y601_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y651_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y701_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y751_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y801_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y851_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y901_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y951_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y1001_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y1051_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y701_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y751_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y801_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y851_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y901_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y951_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y1001_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y1051_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y801_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y851_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y901_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y951_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2101_y1001_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2101_y901_class1.png
inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2101_y951_class1.png

[2] import numpy as np
    import pandas as pd
    import matplotlib.pyplot as plt
    import seaborn as sns
    import cv2
    import glob
    import random
    from sklearn.metrics import classification_report
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import confusion_matrix
    import tensorflow as tf
    from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization
    from tensorflow.keras.optimizers import Adam
    from tensorflow.keras.callbacks import EarlyStopping
    from tensorflow.keras.utils import to_categorical
    import matplotlib.pyplot as plt
    from mpl_toolkits.mplot3d import Axes3D
    from tensorflow.keras.applications import ResNet50
    from tensorflow.keras.models import Model
    from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout

[3] from google.colab import drive
    drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

[4] breast_img = glob.glob('/content/IDC_regular_ps50_idx5/**/*.png', recursive = True)

    for imgname in breast_img[:3]:
        print(imgname)

/content/IDC_regular_ps50_idx5/9174/1/9174_idx5_x1551_y901_class1.png
```

```
/content/IDC_regular_ps50_idx5/9174/1/9174_idx5_x1401_y1101_class1.png
/content/IDC_regular_ps50_idx5/9174/1/9174_idx5_x1501_y801_class1.png
```

```
0s [5] # Split images based on class (IDC negative and IDC positive)
N_IDC = [img for img in breast_img if img[-5] == '0']
P_IDC = [img for img in breast_img if img[-5] == '1']
```

```
0s [6] # Reduce the number of IDC (-) samples to balance the classes
NewN_IDC = N_IDC[:78786]
```

```
0s [7] # Data Preprocessing
non_img_arr = []
can_img_arr = []
```

```
15s [8] # Process IDC (-) images
for img in NewN_IDC:
    n_img = cv2.imread(img, cv2.IMREAD_COLOR)
    n_img_size = cv2.resize(n_img, (50, 50), interpolation=cv2.INTER_LINEAR)
    non_img_arr.append([n_img_size, 0])
```

```
15s [9] # Process IDC (+) images
for img in P_IDC:
    c_img = cv2.imread(img, cv2.IMREAD_COLOR)
    c_img_size = cv2.resize(c_img, (50, 50), interpolation=cv2.INTER_LINEAR)
    can_img_arr.append([c_img_size, 1])
```

```
0s [16] # Combine the processed images and shuffle
breast_img_arr = np.concatenate((non_img_arr[:12389], can_img_arr[:12389]))
random.shuffle(breast_img_arr)

X = np.array([feature for feature, _ in breast_img_arr])
y = np.array([label for _, label in breast_img_arr])

<_array_function__ internals>:180: VisibleDeprecationWarning: Creating an ndarray from ragged nested sequences (which is a list-or-tuple of lists-or-tuples-or ndarrays with different 1
4
```

```
0s [17] # Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

```
2s [18] # Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
2s [19] # Load the pre-trained ResNet50 model without the top (fully connected) layers
resnet = ResNet50(weights='imagenet', include_top=False, input_shape=(50, 50, 3))

# Freeze the layers of the ResNet model
for layer in resnet.layers:
    layer.trainable = False
```

```
0s [21] # Get the output tensor from the ResNet base
x = resnet.output

# Add Global Average Pooling layer
x = GlobalAveragePooling2D()(x)

# Add additional Dense layers with dropout for regularization
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)

x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)

# Final output layer for binary classification (assuming 1 output neuron for sigmoid activation)
output = Dense(1, activation='sigmoid')(x)

# Combine the base ResNet model with the new layers
resnet_model = Model(inputs=resnet.input, outputs=output)

# Compile the ResNet model
resnet_model.compile(optimizer=Adam(learning_rate=0.0001), loss='binary_crossentropy', metrics=['accuracy'])
```

```
0s [22] from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
42m [23] # Augment data during training
train_datagen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    vertical_flip=True,
    fill_mode='nearest'
)

train_generator = train_datagen.flow(X_train, y_train, batch_size=35)

# Train the extended ResNet model
resnet_history = resnet_model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=200, batch_size=35)
```

```
Epoch 172/200
496/496 [=====] - 13s 27ms/step - loss: 0.2587 - accuracy: 0.8967 - val_loss: 0.2598 - val_accuracy: 0.8959
Epoch 173/200
496/496 [=====] - 12s 23ms/step - loss: 0.2593 - accuracy: 0.8984 - val_loss: 0.2577 - val_accuracy: 0.8986
Epoch 174/200
496/496 [=====] - 11s 28ms/step - loss: 0.2602 - accuracy: 0.8973 - val_loss: 0.2608 - val_accuracy: 0.8998
```

```

496/496 [=====] - 14s 20ms/step - loss: 0.2605 - accuracy: 0.8972 - val_loss: 0.2608 - val_accuracy: 0.8998
Epoch 175/200
496/496 [=====] - 12s 24ms/step - loss: 0.2598 - accuracy: 0.8974 - val_loss: 0.2555 - val_accuracy: 0.8987
Epoch 176/200
496/496 [=====] - 13s 27ms/step - loss: 0.2600 - accuracy: 0.8978 - val_loss: 0.2582 - val_accuracy: 0.8988
Epoch 177/200
496/496 [=====] - 13s 27ms/step - loss: 0.2582 - accuracy: 0.8971 - val_loss: 0.2562 - val_accuracy: 0.8970
Epoch 178/200
496/496 [=====] - 13s 27ms/step - loss: 0.2594 - accuracy: 0.8975 - val_loss: 0.2605 - val_accuracy: 0.8964
Epoch 179/200
496/496 [=====] - 14s 27ms/step - loss: 0.2581 - accuracy: 0.8968 - val_loss: 0.2569 - val_accuracy: 0.8990
Epoch 180/200
496/496 [=====] - 11s 23ms/step - loss: 0.2606 - accuracy: 0.8970 - val_loss: 0.2563 - val_accuracy: 0.8983
Epoch 181/200
496/496 [=====] - 14s 28ms/step - loss: 0.2580 - accuracy: 0.8986 - val_loss: 0.2635 - val_accuracy: 0.8917
Epoch 182/200
496/496 [=====] - 14s 28ms/step - loss: 0.2589 - accuracy: 0.8974 - val_loss: 0.2578 - val_accuracy: 0.8991
Epoch 183/200
496/496 [=====] - 11s 23ms/step - loss: 0.2575 - accuracy: 0.8990 - val_loss: 0.2576 - val_accuracy: 0.9007
Epoch 184/200
496/496 [=====] - 11s 22ms/step - loss: 0.2607 - accuracy: 0.8977 - val_loss: 0.2659 - val_accuracy: 0.8909
Epoch 185/200
496/496 [=====] - 12s 24ms/step - loss: 0.2590 - accuracy: 0.8972 - val_loss: 0.2556 - val_accuracy: 0.8997
Epoch 186/200
496/496 [=====] - 14s 28ms/step - loss: 0.2571 - accuracy: 0.8984 - val_loss: 0.2600 - val_accuracy: 0.8997
Epoch 187/200
496/496 [=====] - 11s 23ms/step - loss: 0.2586 - accuracy: 0.8985 - val_loss: 0.2552 - val_accuracy: 0.8992
Epoch 188/200
496/496 [=====] - 13s 27ms/step - loss: 0.2587 - accuracy: 0.8985 - val_loss: 0.2596 - val_accuracy: 0.8980
Epoch 189/200
496/496 [=====] - 14s 28ms/step - loss: 0.2581 - accuracy: 0.8981 - val_loss: 0.2588 - val_accuracy: 0.8990
Epoch 190/200
496/496 [=====] - 14s 28ms/step - loss: 0.2576 - accuracy: 0.8990 - val_loss: 0.2574 - val_accuracy: 0.8998
Epoch 191/200
496/496 [=====] - 14s 29ms/step - loss: 0.2581 - accuracy: 0.8966 - val_loss: 0.2559 - val_accuracy: 0.8978
Epoch 192/200
496/496 [=====] - 14s 28ms/step - loss: 0.2587 - accuracy: 0.8971 - val_loss: 0.2608 - val_accuracy: 0.8966
Epoch 193/200
496/496 [=====] - 11s 23ms/step - loss: 0.2574 - accuracy: 0.8995 - val_loss: 0.2563 - val_accuracy: 0.8988
Epoch 194/200
496/496 [=====] - 13s 27ms/step - loss: 0.2597 - accuracy: 0.8994 - val_loss: 0.2555 - val_accuracy: 0.8967
Epoch 195/200
496/496 [=====] - 11s 23ms/step - loss: 0.2582 - accuracy: 0.8979 - val_loss: 0.2568 - val_accuracy: 0.8939
Epoch 196/200
496/496 [=====] - 12s 24ms/step - loss: 0.2586 - accuracy: 0.8974 - val_loss: 0.2718 - val_accuracy: 0.8873
Epoch 197/200
496/496 [=====] - 11s 23ms/step - loss: 0.2585 - accuracy: 0.8985 - val_loss: 0.2561 - val_accuracy: 0.8998
Epoch 198/200
496/496 [=====] - 11s 22ms/step - loss: 0.2563 - accuracy: 0.9001 - val_loss: 0.2645 - val_accuracy: 0.8939
Epoch 199/200
496/496 [=====] - 14s 28ms/step - loss: 0.2588 - accuracy: 0.8972 - val_loss: 0.2561 - val_accuracy: 0.8951
Epoch 200/200
496/496 [=====] - 12s 24ms/step - loss: 0.2591 - accuracy: 0.8994 - val_loss: 0.2577 - val_accuracy: 0.8967

```

```

[25] # Model Evaluation
resnet_model.evaluate(X_test, y_test)

233/233 [=====] - 5s 17ms/step - loss: 0.2577 - accuracy: 0.8967
[0.25771597027778625, 0.8966909050941467]

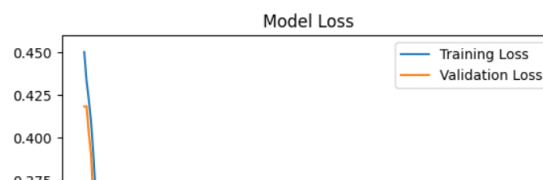
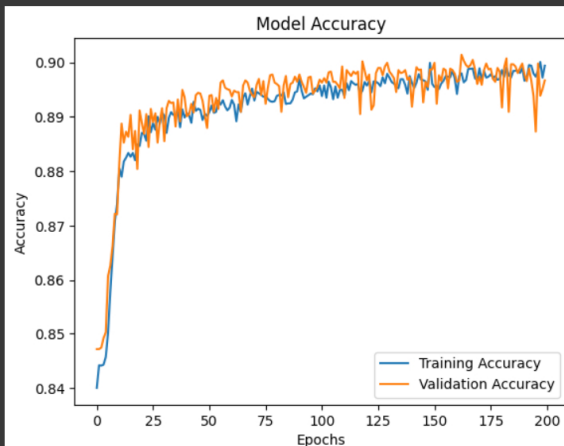
```

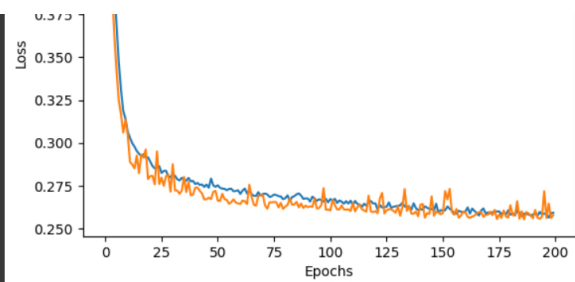
```

[30] # Plotting Accuracy and Loss
plt.plot(resnet_history.history['accuracy'], label='Training Accuracy')
plt.plot(resnet_history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

plt.plot(resnet_history.history['loss'], label='Training Loss')
plt.plot(resnet_history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

```





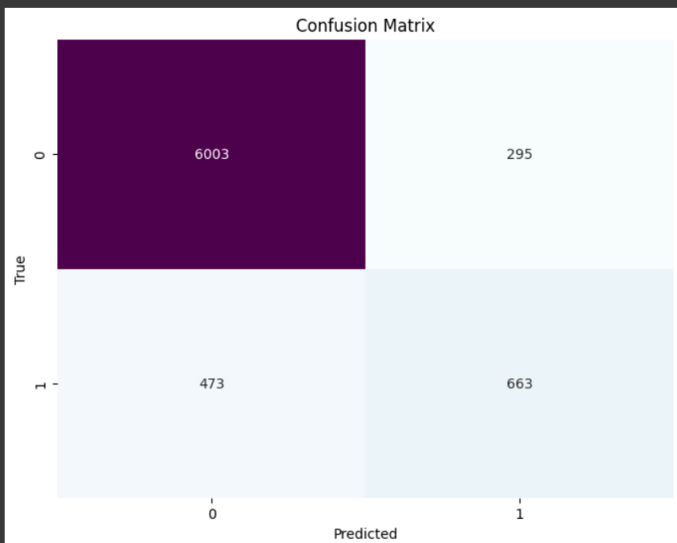
```
[32] # Predict probabilities for the test set
y_pred_prob = resnet_model.predict(X_test)

# Convert probabilities to classes
y_pred = (y_pred_prob > 0.5).astype('int32')

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred)
```

233/233 [=====] - 5s 13ms/step

```
[33] # Plot Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='BuPu', cbar=False)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```

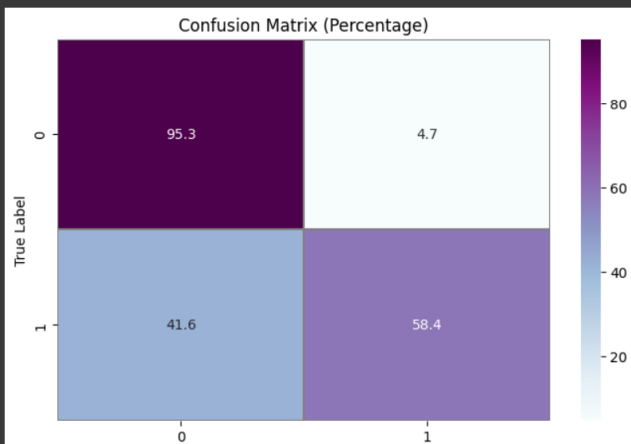


```
[34] from sklearn.metrics import accuracy_score, confusion_matrix

confusion_mtx = confusion_matrix(y_test, y_pred)

# calculate the percentage
confusion_mtx_percent = confusion_mtx.astype('float') / confusion_mtx.sum(axis=1)[:, np.newaxis] * 100

f, ax = plt.subplots(figsize=(8, 5))
sns.heatmap(confusion_mtx_percent, annot=True, linewidths=0.01, cmap="BuPu", linecolor="gray", fmt='.1f', ax=ax)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix (Percentage)")
plt.show()
```



```
[39] # Save the Model  
resnet_model.save("Brest_idc_resnet.h5")
```

/usr/local/lib/python3.10/dist-packages/keras/src/engine/training.py:3103: UserWarning: You are saving your model as an HDF5 file via `model.save()`. This file format is considered legacy. We recommend using the JSON format via `model.save(format='json')` instead.

1



Start coding or generate with AI.



[Colab paid products](#) - [Cancel contracts here](#)

✓ 0s completed at 9:50 PM

